

PROGRAMACIÓN AVANZADA 2026

Entregable 2 – Concurrency

Simulador de restaurante concurrente

INTEGRANTES

Ferreiro · Bruschera

CONTENIDO

Introducción	2
1. Herramientas utilizadas y justificación	2
1.1 Resumen de herramientas	3
1.2 Patrones de diseño concurrente	5
1.3 Cómo se evitan los deadlocks	6
1.4 Visibilidad de los datos compartidos	7
1.5 Excepciones e interrupciones	7
2. Decisiones de funcionamiento, diseño e implementación	8
2.1 Interpretaciones del enunciado	8
2.2 Arquitectura: actores y recursos compartidos	8
2.3 Flujo de cada actor	9
2.4 Cierre ordenado al cumplirse T	10
2.5 Display y log	10
3. Conclusión	11

Introducción

El trabajo es un programa en Java 17, sin dependencias externas, que simula un restaurante. Los clientes, mozos, cocineros y cajeros se ejecutan de forma concurrente, cada uno en su propio hilo, durante un tiempo **T**. Al cumplirse **T** el restaurante cierra de forma ordenada: se cierran las puertas, los clientes que todavía no hicieron su pedido se retiran, los que ya pidieron terminan normalmente, el personal sigue trabajando hasta que no quedan clientes y, por último, terminan todos los hilos y se liberan todos los recursos.

Todas las constantes del enunciado (**T**, **M**, **P**, **Z**, **C**, **Y** y los rangos **TP**, **TM**, **TQ**, **TZ**, **TR**, **TL**, **TC**, **TY**) están en *restaurante.config.Config* como *static final*; cada menú tiene su propio rango **TC**. Cada cambio se notifica a un hilo **Display** a través de una cola, y se registra en un archivo de log de texto.

El grupo tiene dos integrantes, por lo que los requerimientos extra del anexo no forman parte del alcance.

1. HERRAMIENTAS UTILIZADAS Y JUSTIFICACIÓN

El criterio general fue **un mecanismo de sincronización por recurso**, elegido según la forma del problema de ese recurso, en lugar de un lock global que serializaría a todos los actores. La tabla resume cada herramienta, dónde se usa y por qué se eligió.

1.1 Resumen de herramientas

Herramienta	Dónde se usa (clase · método)	Para qué / por qué
EXCLUSIÓN MUTUA		
synchronized (monitor)	<i>Mesa, SalaDeEspera, LogArchivo</i>	Protege la sección crítica de cada mesa y de la sala, donde varios hilos (clientes, mozos y el coordinador) leen y modifican el mismo estado. Junta el estado y las esperas por condición en un solo objeto, y resuelve la condición de carrera entre “el mozo toma el pedido” y “se cierra la mesa”: gana el primero que entra al monitor.
ReentrantLock (justo) + Condition	<i>Cocina: recibirPedido, tomarPlato (hayPlatos.await()), platoListo, cerrar (signalAll)</i>	La operación del cocinero no es “sacar el primero de la cola” sino “buscar, en el primer pedido que lo tenga, un plato sin cocinar y marcarlo”, sin sacar el pedido, porque varios cocineros pueden repartirse los platos de un mismo pedido. Buscar y marcar tiene que ser atómico. El lock justo evita la inanición de un cocinero; la <i>Condition</i> con nombre deja explícito qué se espera. <i>unlock()</i> siempre en <i>finally</i> .
ReentrantReadWriteLock	<i>EstadoRestaurante: notificar</i> (write lock), <i>snapshotActual</i> (read lock)	Muchos escritores (todos los actores notifican sus cambios) y lectores esporádicos (la verificación final y la consulta del estado actual). Bajo el write lock se aplica el cambio, se toma la foto y se encola el evento en un solo paso atómico; así el orden de la cola es el orden real de los cambios.
Semaphore (P·M, justo)	<i>Restaurante.esperarParaEntrar (tryAcquire con timeout), Restaurante.salir (release)</i>	Modela la capacidad del local: entran como máximo P·M clientes y el resto espera afuera. <i>tryAcquire(timeout)</i> en un bucle permite que el cliente que espera afuera note el cierre y se retire. El <i>release</i> está en el <i>finally</i> de <i>Cliente.run</i> : el permiso siempre se devuelve.
Clases atómicas	<i>Restaurante.abierto (AtomicBoolean, compareAndSet), Contadores (AtomicInteger), Pedido.platosPendientes, Caja.recaudacion (AtomicLong), ColaMozos.secuencia</i>	Banderas y contadores que se actualizan desde muchos hilos con una operación indivisible (CAS), sin locks y con visibilidad garantizada. <i>Pedido.platoListo() == 0</i> (un <i>decrementAndGet</i> , ejecutado además con el lock de la cocina) asegura que solo el cocinero que termina el último plato avise a los mozos.
SINCRONIZACIÓN CON BARRERA		
CyclicBarrier (P, acción)	Una por mesa: <i>Mesa.esperarCompaneros (await);</i> acción de barrera <i>Mesa.llamarMozo</i>	“Luego de que los P clientes seleccionan el menú llaman a un mozo” es exactamente una barrera de P partes. La acción la ejecuta una sola vez el último en llegar. Es cíclica: la misma barrera sirve para cada grupo que ocupa la mesa.

Herramienta	Dónde se usa (clase · método)	Para qué / por qué
CountDownLatch	<i>Simulacion.senalCierre</i> (1) y <i>Simulacion.empleadosTerminados</i> (Z+C+Y)	Señales de un solo uso. <i>senalCierre</i> la libera el temporizador en T; <i>GeneradorClientes</i> espera sobre ella con <i>await(espera)</i> en lugar de <i>sleep</i> , así se despierta al instante con el cierre. <i>empleadosTerminados</i> es una barrera de finalización: cada empleado hace <i>countDown</i> en su <i>finally</i> .
PASAJE DE MENSAJES		
PriorityBlockingQueue	<i>ColaMozos</i> (<i>TareaMozo</i>)	Buzón de los mozos. Cualquier mozo libre toma la tarea más prioritaria (los mozos no tienen mesas asignadas). <i>take()</i> los bloquea sin consumir CPU cuando no hay trabajo.
LinkedBlockingQueue	<i>Caja</i> (<i>Pago</i>), <i>EstadoRestaurante.colaEventos</i> (<i>Evento</i>)	Cola única FIFO de pago para los Y cajeros, y cola de notificaciones que consume el Display en orden. Todas las colas son ilimitadas : encolar nunca bloquea (clave para evitar deadlocks, ver 1.3).
POOL DE HILOS		
Executors	<i>Simulacion.correr</i> , <i>newFixedThreadPool</i> (Z, C, Y), <i>newCachedThreadPool</i> (clientes), <i>newSingleThreadExecutor</i> (generador, Display), <i>newScheduledThreadPool</i> (1) (temporizador)	Separan la gestión de hilos de la lógica de cada actor. Personal en cantidad fija: pool fijo. Clientes en cantidad variable: pool <i>cached</i> . Display en un único hilo, para garantizar el orden. <i>shutdown()</i> + <i>awaitTermination()</i> dan un cierre controlado. <i>FabricaHilos</i> nombra los hilos e instala un <i>UncaughtExceptionHandler</i> .
WAIT / NOTIFY / NOTIFYALL		
wait / notifyAll	<i>Mesa: esperarMozo, esperarComida, elegir, tomarPedido, servir, cerrar...</i> · <i>SalaDeEspera: esperarMesa, ingresar, mesaLiberada, cerrar</i>	Esperas por condición dentro de los monitores ("ya vino el mozo", "ya está servida", "el grupo tiene mesa", "se cerró"). Se usa <i>notifyAll</i> y no <i>notify</i> porque en el mismo monitor esperan hilos por condiciones distintas. Todo <i>wait</i> está dentro de un <i>while</i> que vuelve a evaluar la condición.
wait(timeout)	<i>Mesa.elegir</i> (<i>TimeUnit.NANOSECONDS.timedWait</i>)	Elegir el menú tarda TM, pero si el restaurante cierra el cliente tiene que enterarse enseguida. Un <i>sleep</i> no se puede despertar sin interrumpir el hilo; <i>wait(timeout)</i> en un bucle sí se despierta con el <i>notifyAll</i> de <i>Mesa.cerrar()</i> .
COMPLEMENTARIAS		
CompletableFuture	<i>Caja.pagar</i> lo devuelve; <i>Caja.cobrar</i> lo completa; <i>Cliente</i> hace <i>get()</i>	El cliente espera <i>su</i> ticket sin buscarlo en una estructura compartida.
volatile	<i>EstadoRestaurante.inicioNanos</i> , <i>Simulacion.resultado</i> , <i>Display.ultimaSecuencia</i>	Un único escritor y varios lectores: no hace falta exclusión mutua, solo visibilidad.

Herramienta	Dónde se usa (clase · método)	Para qué / por qué
Objetos inmutables	<i>Snapshot</i> (record con <i>List.copyOf()</i> , <i>Evento</i> , <i>TareaMozo</i> , <i>Pago</i> , <i>Ticket</i> , <i>Parametros</i>)	Seguros para hilos por construcción: los mensajes y las fotos del estado se comparten sin locks.
CopyOnWriteArrayList, ThreadLocalRandom	<i>FabricaHilos.hilos</i> ; <i>Aleatorio</i>	Lista de hilos creados recorrible sin lock; un generador aleatorio por hilo, sin estado compartido ni contención.

Alternativas descartadas

- **Un lock global para todo el restaurante:** correcto pero sin concurrencia real. Se prefirió un lock por recurso.
- **BlockingQueue para la cocina:** no permite tomar un plato dejando el pedido en la cola para otro cocinero.
- **acquire() bloqueante en la puerta y sleep al elegir:** el hilo no se enteraría del cierre. Se usaron *tryAcquire(timeout)* y *wait(timeout)*.
- **Phaser:** innecesario, porque las partes de cada mesa son fijas (P) y la acción de barrera de *CyclicBarrier* expresa la regla exacta.
- **System.exit para terminar:** ocultaría hilos colgados. El programa termina porque terminan todos sus hilos no-daemon.

1.2 Patrones de diseño concurrente

Productor-Consumidor (patrón principal)

Es el patrón que estructura toda la simulación. Aparece en cuatro búferes compartidos:

Productores	Búfer	Consumidores
Mesas (acción de barrera: TOMAR_PEDIDO), cocineros (pedido completo: SERVIR), último cliente en irse (LIMPIAR)	<i>ColaMozos</i> — <i>PriorityBlockingQueue<TareaMozo></i>	Z mozos
Mozos (<i>Cocina.recibirPedido</i>)	<i>Cocina</i> — lista FIFO de pedidos con <i>ReentrantLock</i> + <i>Condition</i>	C cocineros
Clientes (<i>Caja.pagar</i>)	<i>Caja</i> — <i>LinkedBlockingQueue<Pago></i> (cola única)	Y cajeros
Todos los actores (<i>EstadoRestaurante.notificar</i>)	<i>LinkedBlockingQueue<Evento></i>	1 Display

Justificación. El enunciado describe literalmente esta situación: los mozos “se quedan esperando” si nadie los llama, existe “una única cola de clientes” para los cajeros y el display “recibe las notificaciones de cada cambio, en una cola, y los va procesando en orden”. El patrón además:

- **desacopla** al productor del consumidor: ningún mozo tiene mesas asignadas y cualquier cajero libre atiende al primero de la fila;
- **evita la espera activa:** *take()* bloquea sin consumir CPU;
- **preserva el orden** (FIFO) y, con un único consumidor, garantiza que el Display muestre los cambios en el orden en que ocurrieron;

- **permite un cierre limpio con *poison pills*** (*TareaMozo.fin()*, *Pago.finDeTurno()*, *Evento.FIN*): el consumidor termina después de procesar todo lo anterior;
- **contribuye a evitar deadlocks**: con búferes ilimitados, producir nunca bloquea.

Monitor

Mesa y *SalaDeEspera* encapsulan su estado y lo protegen con su lock intrínseco; las esperas por condición se hacen con *wait/notifyAll* y ningún otro objeto toca ese estado. Se eligió porque cada mesa tiene **varias condiciones** sobre el mismo estado (eligiendo, esperando mozo, esperando comida, servida, cancelada) esperadas por distintos hilos, y el monitor las concentra en un solo lugar con exclusión mutua.

Barrier

Cada mesa tiene una *CyclicBarrier(P, this::llamarMozo)*: nadie llama al mozo hasta que eligieron los P, y la acción de barrera hace la llamada una sola vez. Se eligió porque es la traducción directa de la regla del enunciado. También se usa una barrera de finalización (*CountDownLatch empleadosTerminados*) para saber cuándo terminó todo el personal.

Working threads

Los actores son *Runnable* ejecutados por pools de *Executors*: cantidad fija de hilos de trabajo para el personal (Z, C, Y) y elástica para los clientes. Separa “qué hace cada actor” de “cómo se crean y terminan los hilos” y da un apagado controlado.

Future / Promise

Caja.pagar devuelve un *CompletableFuture<Ticket>* (la promesa) que completa el cajero en *Caja.cobrar*; el cliente espera el resultado con *get()*. Así cada cliente espera exactamente su cobro, sin estructura compartida adicional.

Read-Write Lock

EstadoRestaurante aplica el problema de lectores y escritores con *ReentrantReadWriteLock*: varios lectores pueden obtener una foto a la vez, y cada escritura es exclusiva.

1.3 Cómo se evitan los deadlocks

Un deadlock requiere una **espera circular**: cada hilo retiene un recurso y espera otro que retiene el siguiente. El diseño elimina esa posibilidad con estas reglas:

1. **No se anidan locks entre recursos.** El monitor de una mesa nunca toma el de otra mesa ni el de la sala; la sala lleva su propia lista de mesas libres y no entra al monitor de las mesas. El mozo avisa a la sala (*SalaDeEspera.mesaLiberada*) después de salir del monitor de la mesa, y el cocinero encola SERVIR después de soltar el lock de la cocina.
2. **Orden fijo barrera → mesa.** Sin contar el lock hoja y las colas (puntos 3 y 4), la única cadena entre locks es la acción de barrera (*Mesa.llamarMozo*), que corre con el lock interno de la *CyclicBarrier* y entra al monitor de la mesa. Nunca se toma en el orden inverso: *Mesa.terminarLimpieza()* hace el *reset()* de la barrera antes de entrar al monitor, y *Mesa.esperarCompaneros()* no es *synchronized* (si lo fuera, el primer cliente esperaría en la barrera reteniendo el monitor y los demás no podrían llegar a ella).
3. **Lock hoja.** El write lock de *EstadoRestaurante* se toma teniendo otro lock (por ejemplo desde *Mesa.servir* o desde la cocina), pero mientras se lo tiene no se toma ningún otro: solo se modifica

la vista y se agrega a una cola ilimitada. Un lock que siempre es el último de la cadena no puede cerrar un ciclo.

4. **Colas ilimitadas.** *put/add* nunca bloquean, así que encolar con un monitor tomado (*Mesa.llamarMozo* → *ColaMozos.encolar*) no deja a nadie esperando. Las únicas esperas indefinidas son *take()*, *await()*, *wait()* y el *get()* del pago, y en ellas el hilo no retiene ningún otro lock (*wait* y *Condition.await* liberan el propio).
5. **La barrera siempre se completa.** Los P clientes de un grupo llegan siempre a *await()*, incluso si la mesa se canceló por el cierre; la acción de barrera ve *cancelada* y no llama al mozo.
6. **Esperas con límite.** La puerta usa *tryAcquire* con timeout (*Config.ESPERA_PUERTA_MS*). El coordinador espera pools y latches con un máximo (*Config.TIMEOUT_CIERRE_MS*); si se superara, vuelca la pila de todos los hilos al log (*Simulacion.volcarHilos*) y fuerza con *shutdownNow*. Es una red de seguridad, no parte del funcionamiento normal.

Inanición. Las colas de caja, cocina y eventos son FIFO (la de mozos, FIFO dentro de cada prioridad) y el *Semaphore* y el *ReentrantLock* de la cocina son justos. Las prioridades de los mozos están acotadas (ver 2.1). **Bloqueo activo:** no hay reintentos que se cedan el paso mutuamente; el único reintento es el *tryAcquire* de la puerta, que no cede nada. **Self-deadlock:** todos los locks usados son reentrantes y cada cliente toma un único permiso del semáforo.

1.4 Visibilidad de los datos compartidos

Para que ningún hilo lea un valor viejo de la caché, todo dato compartido se accede a través de una herramienta que establece una relación *happens-before* del modelo de memoria de Java:

Acción del hilo que escribe	Acción del hilo que lee	Dónde
Salir de un bloque <i>synchronized</i>	Entrar al mismo monitor	<i>Mesa, SalaDeEspera</i>
<i>unlock()</i>	<i>lock()</i> del mismo lock	<i>Cocina, EstadoRestaurante</i>
<i>put()</i> / <i>add()</i> en una <i>BlockingQueue</i>	<i>take()</i> del mismo elemento	Mozos, cajeros, Display
<i>release()</i>	<i>tryAcquire()</i>	Puerta
<i>countDown()</i>	Retorno de <i>await()</i>	Latches
<i>complete()</i> del <i>CompletableFuture</i>	Retorno de <i>get()</i>	Pago
Escritura de un <i>volatile</i> o atómico	Lectura de esa variable	<i>abierto</i> , contadores, <i>inicioNanos</i>

Por eso, por ejemplo, *Plato.estado* no necesita *volatile*: solo se lee y escribe con el lock de la cocina tomado. Los mensajes (*TareaMozo*, *Pago*, *Evento*) son inmutables y se publican a través de colas concurrentes (publicación segura).

1.5 Excepciones e interrupciones

- **InterruptedException:** se captura, se restaura el flag con *Thread.currentThread().interrupt()* y el hilo termina ordenadamente.
- **Los recursos se liberan en finally:** *Cliente* devuelve el permiso de la puerta; *Mozo*, *Cocinero* y *Cajero* notifican que terminaron y hacen *countDown*; *Cocina* y *EstadoRestaurante* hacen *unlock*.
- **BrokenBarrierException:** cancela la ocupación de la mesa (*Mesa.marcarCancelada*); la barrera se resetea al limpiar la mesa.
- **ExecutionException** del futuro de pago se informa como error del cliente.

- **El coordinador no abandona el cierre:** si se lo interrumpe, lo anota, sigue esperando y restaura la interrupción al final.
- **El Display no muere** por un error al procesar un evento: lo registra y sigue con el siguiente. Las excepciones no capturadas de cualquier hilo las registra el *UncaughtExceptionHandler* en consola y log.

2. DECISIONES DE FUNCIONAMIENTO, DISEÑO E IMPLEMENTACIÓN

2.1 Interpretaciones del enunciado

- **“Realizó su pedido”** significa que el mozo ya lo tomó (*Mesa.tomarPedido()* ya se ejecutó). Aunque la mesa haya elegido y llamado al mozo, si cierra antes de que el mozo llegue, sus clientes se retiran sin pedir; “cuando el mozo llega hacen el pedido”.
- **Capacidad P·M.** Un cliente ocupa un lugar desde que entra hasta que sale del local, después de pagar. Una mesa sucia no reduce la capacidad: los que entran esperan en la sala hasta que haya mesa libre.
- **Grupos.** Los clientes se agrupan de a P en orden de entrada; los grupos completos reciben mesas libres en orden FIFO. Un grupo incompleto se queda esperando, como pide el enunciado.
- **Prioridades del mozo: SERVIR > TOMAR_PEDIDO > LIMPIAR** (*TipoTarea*), y FIFO dentro de la misma prioridad (número de secuencia en *TareaMozo.compareTo*). Servir es lo más urgente porque la comida ya está lista; tomar pedidos alimenta a la cocina; limpiar puede esperar. El riesgo de inanición de LIMPIAR está acotado: como mucho hay una tarea SERVIR o TOMAR_PEDIDO por mesa ocupada y una mesa sucia no genera nuevas, así que al agotarse los mozos limpian. La *poison pill FIN* tiene la prioridad más baja, de modo que en el cierre primero se completa toda la limpieza.
- **Varios cocineros por pedido.** “Un cocinero libre toma del primer pedido de la cola algún menú sin cocinar”: si el primer pedido tiene varios platos pendientes, otro cocinero toma el siguiente plato del mismo pedido; solo cuando no quedan pendientes se pasa al segundo pedido. El que completa el último plato es el único que avisa a los mozos.
- **Un plato por cliente;** cada cliente paga el suyo.
- **La mesa queda sucia cuando se va el último cliente** y queda LIBRE cuando un mozo termina de limpiarla. Las mesas canceladas por el cierre también se limpian, así que al final todas quedan libres.

2.2 Arquitectura: actores y recursos compartidos

Cada actor es un hilo; los recursos compartidos son objetos pasivos que encapsulan su propia sincronización. El código se organiza en paquetes: *actores*, *recursos*, *modelo*, *estado*, *display*, *simulacion* y *config*.

Actor (hilo)	Pool	Recursos compartidos que usa
<i>Cliente</i>	<i>newCachedThreadPool</i>	Puerta (<i>Semaphore</i>), <i>SalaDeEspera</i> , <i>Mesa</i> , <i>Caja</i> , <i>ColaMozos</i> (LIMPIAR)
<i>Mozo ×Z</i>	<i>newFixedThreadPool(Z)</i>	<i>ColaMozos</i> , <i>Mesa</i> , <i>Cocina</i> , <i>SalaDeEspera</i>

Actor (hilo)	Pool	Recursos compartidos que usa
Cocinero ×C	<i>newFixedThreadPool(C)</i>	Cocina, ColaMozos
Cajero ×Y	<i>newFixedThreadPool(Y)</i>	Caja
GeneradorClientes	<i>newSingleThreadExecutor</i>	Pool de clientes, <i>senalCierre</i>
Display	<i>newSingleThreadExecutor</i>	Cola de eventos, <i>LogArchivo</i>
Temporizador	<i>newScheduledThreadPool(1)</i>	Libera <i>senalCierre</i> a los T ms
Coordinador	hilo que llama a <i>Simulacion.ejecutar</i>	Todos: ejecuta el cierre ordenado

Restaurante agrupa los recursos (puerta, bandera *abierto*, mesas, sala, cocina, caja, cola de mozos y contadores). Todos los actores informan sus cambios a *EstadoRestaurante*, que es el único punto de notificación.

Estados de una mesa (*EstadoMesa*), protegidos por su monitor:

LIBRE → ELIGIENDO → (barrera) ESPERANDO_MOZO → TOMANDO_PEDIDO → ESPERANDO_COMIDA → SIRVIENDO → COMIENDO → SUCIA → LIMPIANDO → LIBRE
Si cierra antes de TOMANDO_PEDIDO: la ocupación se cancela, los clientes se retiran sin pedir y la mesa pasa a SUCIA.

2.3 Flujo de cada actor

Cliente (*Cliente.run*): espera afuera con *Restaurante.esperarParaEntrar()*; entra con *SalaDeEspera.ingresar()* y se suma al grupo en formación; espera mesa con *SalaDeEspera.esperarMesa()* (*wait*); se sienta (*Mesa.sentarse*), elige el menú al azar durante TM (*Mesa.elegir*) y espera a sus compañeros en la barrera (*Mesa.esperarCompaneros*); el último llama al mozo. Espera al mozo (*Mesa.esperarMozo*): si devuelve *false* la mesa se canceló y se retira sin pedir. Si no, espera la comida (*Mesa.esperarComida*), come durante TQ, paga (*Caja.pagar + get()*) y se retira (*Mesa.retirarse*); si es el último de la mesa, encola LIMPIAR. En el *finally* devuelve el permiso de la puerta.

Mozo (*Mozo.run*): hace *take()* sobre *ColaMozos* y atiende la tarea recibida.

- **TOMAR_PEDIDO**: *Mesa.tomarPedido()*; si devuelve *null* (el cierre ganó la carrera) descarta la tarea. Si no, anota durante TZ, marca *Mesa.pedidoEnCocina()* y entrega con *Cocina.recibirPedido()*.
- **SERVIR**: *Mesa.empezarAServir()*, espera TR y *Mesa.servir()*, que despierta a los clientes.
- **LIMPIAR**: *Mesa.empezarLimpieza()*, espera TL, *Mesa.terminarLimpieza()* y luego, fuera del monitor, *SalaDeEspera.mesaLiberada()*.
- **FIN**: termina (poison pill).

Cocinero (*Cocinero.run*): *Cocina.tomarPlato()* (espera en la *Condition* si no hay platos), cocina durante el TC del menú y llama a *Cocina.platoListo()*; si el pedido quedó completo, encola SERVIR. Termina cuando *tomarPlato()* devuelve *null* (cocina cerrada y sin platos pendientes).

Cajero (*Cajero.run*): *Caja.tomar()*, cobra durante TY y *Caja.cobrar()* completa el futuro del cliente. Termina con la poison pill *Pago.finDeTurno()*.

2.4 Cierre ordenado al cumplirse T

El coordinador (*Simulacion.correr*) espera *senalCierre* y ejecuta el cierre en este orden:

1. **Se cierran las puertas:** *Restaurante.cerrarPuertas()* pone *abierto = false* con *compareAndSet*. El generador, despertado por el latch, deja de crear clientes; los que esperan afuera salen de su bucle *tryAcquire* y se retiran. Si alguno obtuvo permiso en ese instante, *SalaDeEspera.ingresar()* verifica *abierto* dentro del monitor y lo rechaza. Desde ahí la sala tampoco asigna mesas nuevas.
2. **Se cierra la sala:** *SalaDeEspera.cerrar()* hace *notifyAll*; los grupos incompletos y los que esperaban mesa se retiran sin pedir.
3. **Se cierra cada mesa:** *Mesa.cerrar()* cancela la ocupación si el pedido no fue tomado y hace *notifyAll*: los que eligen cortan la espera, todos atraviesan la barrera (que no llama al mozo) y los que esperaban al mozo se retiran.
4. **Los que ya pidieron terminan normalmente** (comen, pagan y se van); el coordinador hace *shutdown()* + *awaitTermination()* del pool de clientes.
5. **El personal termina:** Z poison pills a los mozos (prioridad mínima, así primero limpian lo pendiente), *Cocina.cerrar()* con *signalAll* a los cocineros e Y poison pills a los cajeros. El coordinador espera con *empleadosTerminados.await()*.
6. **Se apagan todos los pools** (*shutdown* + *awaitTermination*).
7. **El Display vacía su cola:** se encola *Evento.FIN*; el Display procesa todo lo anterior y termina. Luego se verifican invariantes de cierre (nadie adentro, mesas libres, colas vacías, ningún hilo vivo) y se cierra el log.

Carrera “el mozo toma el pedido” contra “se cierra la mesa”. *Mesa.tomarPedido* y *Mesa.cerrar* son *synchronized* sobre la misma mesa, así que se ejecutan una tras otra. Si entra primero *tomarPedido*, queda *pedidoTomado = true* y el cierre no cancela nada. Si entra primero *cerrar*, queda *cancelada = true* y *tomarPedido* devuelve *null*. No existe un estado intermedio.

2.5 Display y log

- **Un solo punto de notificación.** Cada actor llama a *EstadoRestaurante.notificar(actor, descripción, cambio)*. Bajo el write lock se aplica el cambio sobre una *Vista* privada, se arma un *Snapshot* inmutable y se encola el *Evento* con un número de secuencia. Así cada evento lleva el estado completo del restaurante, con todos los actores, **del momento en que se mandó la notificación**, aunque el Display lo procese más tarde. Mostrar el estado nunca requiere tomar los locks de las mesas o de la cocina.
- **Un solo consumidor.** El hilo *Display* corre en un *newSingleThreadExecutor*, consume la *LinkedBlockingQueue<Evento>* en orden FIFO y verifica que las secuencias sean crecientes. Por cada evento escribe la acción y el estado completo en el log y lo muestra por consola.
- **Después de T el Display sigue.** Su poison pill se encola recién cuando terminaron todos los actores, de modo que muestra todos los cambios pendientes y finaliza cuando la cola queda vacía.
- **Log.** *LogArchivo* escribe en *logs/simulacion-<fecha>.log*; sus métodos son *synchronized* porque, además del Display, escriben el coordinador y el manejador de excepciones.

3. CONCLUSIÓN

Producto desarrollado

El simulador cumple los requerimientos funcionales y técnicos del enunciado: cuatro roles que se ejecutan de forma concurrente, cierre ordenado en T con todas sus reglas, display en orden con el estado completo de cada momento y log en archivo. Usa todas las herramientas obligatorias (*synchronized*, *ReentrantLock*, *ReentrantReadWriteLock*, *Semaphore* y clases atómicas para exclusión mutua; *CyclicBarrier* y *CountDownLatch* como barreras; *BlockingQueue* para pasaje de mensajes; *Executors*; *wait/notifyAll*) y se apoya en los patrones Productor-Consumidor, Monitor, Barrier, Working threads, Future-Promise y Read-Write Lock. Al terminar cada corrida se verifica que no queden clientes, mesas ocupadas, mensajes sin procesar ni hilos vivos.

Aprendizaje sobre concurrencia

- **Lo más difícil fue el cierre, no el caso normal.** Terminar hilos bloqueados en esperas distintas (afuera, en la sala, en la barrera, esperando al mozo, en *take()*) obliga a responder, para cada espera, quién la despierta y cómo se entera el hilo de que tiene que irse. Eso nos llevó a preferir esperas interrumpibles o con timeout (*wait(timeout)*, *tryAcquire(timeout)*, *await(timeout)*) sobre *sleep* o *acquire*.
- **La herramienta se elige por la forma del problema:** una barrera para “esperar a que todos elijan”, un monitor cuando hay varias condiciones sobre el mismo estado, *ReentrantLock* con *Condition* cuando la operación no es un simple *take*, y colas cuando los actores solo necesitan pasarse trabajo.
- **Los deadlocks se previenen por diseño** más fácilmente que depurándolos: no anidar locks, fijar un orden cuando no se puede evitar la cadena, usar locks hoja y no bloquear nunca reteniendo un lock.
- **Las condiciones de carrera se resuelven ubicando la decisión en una sola sección crítica**, como la carrera entre tomar el pedido y cerrar la mesa: si ambas operaciones pasan por el mismo monitor, el resultado es siempre coherente.
- **La visibilidad no es automática:** hay que razonar qué relación *happens-before* publica cada dato; la inmutabilidad y las colas concurrentes simplifican mucho compartir información entre hilos.
- **Un sistema concurrente es no determinista:** dos ejecuciones nunca dan el mismo orden, por lo que conviene verificar propiedades (invariantes) en lugar de comparar salidas.